Received: 1 September 2019 | Revised: 20 April 2020 | Accepted: 9 June 2020

DOI: 10.1111/exsy.12608

ORIGINAL ARTICLE

Expert Systems WILEY

Automating test oracles from restricted natural language agile requirements

Maryam Imtiaz Malik | Muddassar Azam Sindhu | Akmal Saeed Khattak | Rabeeh Ayaz Abbasi | Khalid Saleem

Department of Computer Science, Quaid-i-Azam University, Islamabad, Pakistan

Correspondence
Muddassar Azam Sindhu, Department of Computer Science, Quaid-i-Azam University, Islamabad 45320, Pakistan.
Email: masindhu@qau.edu.pk

Abstract

Manual testing of software requirements written in natural language for agile or any other methodology requires more time and human resources. This leaves the testing process error prone and time consuming. For satisfied end users with bug-free software delivered on time, there is a need to automate the test oracle process for natural language or informal requirements. The automation of the test oracle is relatively easier with formal requirements, but this task is difficult to achieve with natural language requirements. This study proposes an approach called *Restricted Natural Language Agile Requirements Testing* (ReNaLART) to automate the test oracle from restricted natural language agile requirements. For this purpose, it uses an existing user story template with some modifications for writing user stories. This helps in identifying test input and expected output for a user story. For comparison of expected and observed outputs it makes use of a regex pattern and string distance functions. It is capable of assigning different types of verdicts automatically depending upon the similarity/dissimilarity between observed and expected outputs of user stories. ReNaLART is validated using several case studies of different domains, namely, OLX Pakistan, Mental Health Tests, McDelivery Pakistan, BlueStacks, Power Searching with Google, TensorFlow Playground, w3Schools 2018 offline and Touch'D. It revealed several faults in five of the above listed eight applications. Plus, the proposed test oracle on an average took 0.02 s for test data generation, expected output generation and verdict assignment. Both these facts show the fault revealing effectiveness and efficiency of ReNaLART.

KEYWORDS

automated test oracle, automated test-driven development, behaviour-driven development, restricted natural language, agile requirements

1 | INTRODUCTION

Starting from the classic waterfall process model to the more recent agile process models, the focus of development has shifted from clearly gathered and well-documented requirements to handle creeping requirements with less focus on spending ample time on documenting requirements. Since the agile process caters for changing requirements, this poses a challenge for testers to test functionality corresponding to these changing requirements in an automated and efficient manner. As natural language is used widely as a tool in the software industry for documenting

requirements even in the agile paradigm, therefore, this compounds the challenge of automating the test oracle from these requirements because automating the test oracle is relatively easier with formal requirements.

The importance of automating the test oracle for natural language-based agile requirements becomes even more significant because of studies which show that a large number of errors are rooted in badly written or inadequately tested requirements (Alagar & Periyasamy, 2011). In addition, the errors rooted in requirements become more expensive to remove with the passage of each development phase (Sommerville & Sawyer, 1997).

Due to the challenge of changing requirements it is difficult for testers to test the requirements manually and to satisfy the customers with changing requirements. Therefore, an automated test oracle is desirable to test agile requirements for an error-free software.

The main purpose of this study is to present an approach which automates the test oracle for restricted natural language agile requirements by following the definition of Shahamiri, Kadir, and Mohd-Hashim (2009).

The goal of testing requirements in general and agile requirements in particular is to check that all requirements are implemented in accordance with the customer demands. Like testing of requirements written in other paradigms, testing of agile requirements can be performed either manually, semi-automatically or fully automatically.

Most literature on testing natural language requirements only contains the approach for test case generation from natural language requirement whereas the test oracle process is either manually performed or the JUnit tool is used by invoking its assert functions. Manual testing of agile requirements has several drawbacks that are

- More time consuming.
- More human effort required.
- More error prone.

Therefore, there is a need to have an automated test oracle for natural language software requirements.
This study has the following research contributions in this context:

1. It provides a new template for writing restricted natural language agile requirements which are used for automating the test oracle.
2. It provides a mechanism to automatically generate test data and corresponding expected output from requirements written in the template proposed in Point 1 above.
3. Finally, it provides a mechanism to automatically assign test verdicts corresponding to each requirement.

The rest of the article is organized as follows: Section 2 provides a background of various aspects of research presented in this study along with providing the related work of the field. Section 4 describes the proposed approach which we call *Restricted Natural Language Agile Requirements Testing* (ReNaLART). Section 5 briefly discusses the working of ReNaLART with an example. Section 6 gives the experimental setup to evaluate ReNaLART. Section 7 provides the results obtained after applying ReNaLART on case studies along with their analysis. Section 8 presents the effectiveness of ReNaLART in terms of execution time and compares this with existing approaches. Finally, we give conclusions and some future directions of research in Section 9.

2 | BACKGROUND AND RELATED WORK

In this section, we review some literature which is relevant to the research presented in this study. This includes literature on user story templates for writing agile requirements, test case generation techniques from requirements, oracle strategies used in software testing for natural language requirements and finally review some literature about testing tools following the agile paradigm of software development. We begin by discussing user stories and the templates available in the literature to write user stories.

2.1 | User stories

User stories are used to write software requirements in the agile development methodology. There are different templates available for writing user stories such as Cohn (2004) and Connextra (Role-feature-reason, 2017) templates. Each template has some distinctive features that distinguishes it from the rest. Most of the templates consist of *role*, *mean*, *action* and *benefit* parts. Some of these templates provide the flexibility of making some of these parts optional. For example, the Cohn's template allows the *benefit* part to be optional. Baumer and Geierhos (2016) proposed an approach having no restriction of using any specified template for user stories. The main goal of this approach was to detect a missing argument and provide suggestion(s) to complete an incomplete requirement. This involved gathering of similar user stories from different websites

to complete the missing user story. Since this approach did not require a specific template, therefore, the authors of this study gave the idea of making use of a template for user stories containing role, mean and action parts to ensure the completeness of requirements.

Landhausser and Genaid (2012) proposed an approach that facilitates the developer to connect user stories and acceptance tests with the code. This was done via ontologies. Querying the ontology enabled to avoid duplicate tests in the knowledge base thus saving rework and effort.

In Pandit and Tahiliani (2015), authors discuss a template of user stories and acceptance tests. They performed testing with the help of use case diagrams. However, in our proposed approach testing is performed on requirements that are written in restricted natural language and a use case diagram is not required for testing.

2.2 | Oracle

A test oracle is a mechanism used in software testing which generates expected output corresponding to a test input, compares it with the observed output of that test input and finally assigns a test verdict (Shahamiri et al., 2009).

According to Hoffman (1998), the test oracle is the process of generating expected output. Mathur (2013) suggests that an oracle in software testing is a process for comparison of expected and observed behaviour of software.

A *pass* and a *fail* are two primary types of verdicts that are commonly used by most of the currently used oracles. A test pass verdict is assigned if the expected and observed outputs are the same, otherwise, a test fail verdict is assigned. Moreover, a test warning, a test inconclusive and a test timeout are some other types of verdicts used by oracles of certain testing tools. Most software testing tools especially those used for agile requirements testing assign verdicts by making use of the JUnit tool's assert function (see Tahchiev, Leme, Massol, & Gregory, 2010).

Now, we review some literature for test oracle generation. Goffi, Gorla, Ernst, and Pezzè (2016) generate an oracle for testing an exception in a program. The comments related to an exception are taken from its Java doc. The predicates and subject are extracted from comments and these comments are converted into a Java expression through a condition translator module that contains conditional and relational operators or Boolean values depending upon the pattern identified from the predicate. For example, in a comment containing a *not*, a false Boolean value is used in the Java expression. This oracle needs a method, a Java expression and an expected exception along with each test input for the comparison of an expected and an observed exception. A fail verdict is assigned if expected exceptional behaviour is not found to be the same as the observed exceptional behaviour or if no exception is present in the expected list, but an exception occurs upon executing the System Under Test (SUT). A pass verdict is assigned otherwise. Goffi et al. (2016) also give the motivation for an approach which creates test oracles from requirements expressed in natural language and not just comments. This according to them should be able to give verdicts other than just a pass and a fail.

Barr, Harman, McMinn, Shahbaz, and Yoo (2015) discuss different types of oracles by categorizing them into specified, derived, implicit or another type. These may still require human involvement for ascertaining whether the observed behaviour is as expected or not. One such oracle is a specified test oracle which makes use of a formal specification for testing the SUT. For example, the TestEra (Khurshid & Marinov, 2004) tool performs testing by taking the Java method as an input along with the formal representation of its pre- and post-conditions. It also requires a bound on the input size for automatically generating test inputs within this bound. Barr et al. (2015) also provide different ways to provide the formal representation. It can either be model based or specification based. In the model-based representation the system failure is determined by matching system behaviour against the behaviour of the model. In the specification-based scenario, assertions are used as a specified test oracle. This oracle can return two types of verdicts: a pass or a fail depending upon the results obtained after the comparison of expected and observed outputs. Moreover, Barr et al. (2015) also discuss state transition systems and algebraic specification as a way to specify system behaviour formally. They also discuss another type of oracle termed as the 'derived oracle'. This oracle extracts information from different resources, for example, textual documentation, program invariants and so on. However, the criteria for checking the program behaviour depends upon the selection of an artefact. The derived oracle makes use of a pseudo oracle, metamorphic testing, and regression testing. They further mention that testing of textual document artefacts is performed by converting these documents into some kind of a formal specification or making use of controlled natural language. ReNaLART also uses the later technique of controlled natural language for automating the test oracle process from agile requirements.

Afshan, McMinn, and Stevenson (2013) propose an approach to limit the impact of manual oracles. They generate test inputs automatically using a probabilistic language model. The model assigns a low score to all those words in a requirement which are not easily readable by a human. The authors compute the human oracle cost and show that it is improved by automatically generating readable input strings. On the contrary, ReNaLART computes the oracle cost in terms of time taken for the test oracle creation and execution parts. Moreover, it does not make use of a probabilistic model or a human oracle.

Mayer and Guderlei (2004) discuss commonly used statistical oracles when the input and expected outputs are randomly generated and then, compared with some features of observed output by applying different statistical methods. This oracle assigns a test pass, and a test fail verdict. The statistical oracle presents result using multiple test cases rather than a single test case. However, our approach does not have a random test input and assigns a test verdict based on a single test case.

Peters and Parnas (1998) discuss different types of documentations out of which one generates an oracle from a program description by converting it into a tabular form containing mathematical expressions and predicates. However, this lacks ease of use due to the complex description of tables.

Hoffman (1998) categorize oracles on the basis of how expected results are generated. They categorize, a *true* oracle to be one which is implemented independently of the SUT, and then is used to evaluate the results. A *consistent* oracle on the other hand uses the result of one test as an oracle for a subsequent test. These oracles usually are used for evaluating the changes in a subsequent version of a software. The *stochastic* and *sampling* oracles are two other types of oracle which use statistical computation for the selection of input values and then generate the corresponding expected result. However, in the sampling oracle, the random generator method is not included. The authors only categorize oracles based on the expected output generation method used in them. However in ReNaLART, the oracle is also used for the comparison of expected and observed outputs as in Shahamiri et al. (2009).

Another oracle named as the *Heuristic* oracle is discussed by Hoffman (1998). It selects some values from the input for testing and based on their results tests the remaining values. The selection of initial values is crucial as the correct prediction of remaining test results depends on the outcome of the initially selected values. This oracle is useful for applications having a large number of records and testing each record is not possible. Furthermore, this type of oracle is only feasible for those systems where a value or a range of values can be selected for testing and their results work as a heuristic for all other remaining records. This dependency is not present in ReNaLART.

Another essential feature of automated testing with an automated test oracle consists of automatically extracting expected output from the requirements and making it available to the oracle for comparison with the observed output. In Shahamiri et al. (2009), the level of automation in five different types of oracles is compared. Artificial Neural Network (ANN) and decision table-based oracles generate expected output manually (see Lucca, Fasolino, Faralli, & Carlini, 2002) and make use of semi-automated tools for analysis of input and output. The remaining AI Planner Test Oracle (see Memon, Pollack, & Soffa, 2000), Info Fuzzy Network (IFN) (see Last, Friedman, & Kandel, 2004), N-Version Diverse and M-Model Program Testing (see Manolache & Kourie, 2001) employ automated tools. However, no automated tool is used for comparison of expected and observed outputs for natural language requirements. The decision table-based approach also employs a manual comparison despite being a formal approach. However, in ReNaLART, the processes of expected output generation, the observed and expected output comparison, and verdict assignment, all are done automatically.

2.3 | Test case generation from requirements

In this subsection, we review some approaches available in the literature for test case generation from natural language requirements.

Zhang, Yue, Ali, Zhang, and Wu (2014) generate test cases automatically from a restricted natural language use case by applying some predefined coverage criteria. These test cases are generated through use case (UC) and and test case (TC) meta-models. This approach gives motivation for automatic generation of test cases from natural language after applying some restrictions on writing natural language requirements and using keywords. However, there is no mention of a method which automatically compares expected and observed outputs in this work. This task is performed automatically in ReNaLART.

Yue, Ali, and Zhang (2015) generated the executable test cases automatically from restricted natural language use cases. The authors of this article extend the approach of Zhang et al. (2014) by incorporating an oracle verification part which compares observed and expected outputs through assertions. ReNaLART does not use assertions for this purpose and a verdict is assigned automatically in our work.

Carrera, Iglesias, and Garijo (2014) propose an approach for testing of real multi-user systems involving user stories and acceptance criteria. The test cases in this case are generated from acceptance criteria. The Beast tool (Carrera et al., 2014) generates classes for executing test cases, but these classes need to be implemented by the tester. The JUnit tool is used for comparison of outputs. However, our approach does not have any dependency on the JUnit tool for verdict assignment.

Sarmiento, do Prado Leite, and Almentero (2014) propose an approach for automatically generating test cases from natural language. They create an activity diagram from requirements for the generation of test cases to produce a test scenario. This requires human intervention for test data generation. They do not consider automatic verdict assignment in their approach which is addressed in our approach without human intervention.

Aichernig et al. (2014) propose an approach in which textual requirements are used as an input for test case generation. However, for generating test cases, the textual requirements are formalized, which are checked for consistency both in the informal and formal versions. Moreover, two types of verdicts: pass or fail are assigned after output comparison. ReNaLART does not require formalizing of requirements for test case generation and test verdict assignment which saves time and cost.

Wang, Pastore, Goknil, Briand, and Iqbal (2015) propose an approach for automatically generating test cases from the domain model and use case specification. The use cases are written in Restricted Use Case Modelling and the domain model is manually created. The completeness of the domain model is checked automatically. Moreover, use case behaviour is extracted using natural language processing (NLP) pipeline which identifies the parts of speech (POS) tags and keywords from the use case scenarios. The approach requires manual generation of object constraint

language (OCL) constraints by engineers. The test model is created by using OCL constraints, domain model and use case specification which are further used in the generation of test inputs and test cases. The approach also automatically generates post condition from specification using NLP pipeline which is then translated into a scripting language using a mapping table for test oracle generation. This approach takes 12 min on an average for the generation of a test case. However, our approach takes less time for test case generation which will be discussed in Section 8. Moreover, our approach generates test cases without designing domain model and specifying OCL constraints. The test oracle process in ReNaLART does not require the conversion of OCL constraints and thus results in reduced time.

Wang, Pastore, and Briand (2017) propose another approach for system testing in which test cases are generated automatically from use case specifications. The approach generates executable test cases from natural language requirements in combination with timed automata which is again a type of formalism.

Elghondakly, Moussa, and Badr (2015) propose a technique for generating test cases. The test cases are generated for waterfall and agile requirements. In this technique, first the requirement type is identified and then the relevant test case, test data generation, validation and optimization techniques are applied. The test data generation, optimization and validation techniques are applied on user stories if the requirements are written for an agile process whereas functional and non-functional requirements are used for the waterfall model. POS tags are identified using parsers and only words having the verb POS tag are retained for test data generation. This technique uses symbolic execution using these verbs. The results of this technique, before and after applying test validation and optimization methods are compared using three case studies. ReNaLART does not require symbolic execution and POS tags extraction for test data and expected output generation which saves time and effort.

Rane (2017) discuss an approach for generating test cases from agile requirements. Test cases are either generated from user stories or from an activity diagram. The POS tags are applied to extract nouns and verbs from the functionality part of the user story. The extracted words are then matched with the test scenario description. The test scenario description is required to be provided with each user story. In the case of an exact match test cases are generated, otherwise, synonyms and lemmas of nouns and verbs are extracted from user stories and matched with test scenarios to generate test cases. Active and passive voice are identified using POS tagging to generate test cases from an activity diagram. These are then collected in a frame by using dependency parsing. These can then be used for discovering paths from an activity graph and the discovered paths can be used to generate relevant test cases. The approach proposed by Rane (2017) has a limitation that each user story must have a test scenario description for the generation of test cases. Furthermore, the approach did not extract expected output from the user story. ReNaLART does not require to have a test scenario description along with each user story and also assigns verdicts automatically which is not performed by Rane (2017).

Sneed (2007) discuss an approach for test case generation and extraction from natural language requirements. In the first step, objects are identified by the extraction of nouns and keywords. In the next step, those sentences are selected which contain objects and these sentences are further classified as state, action and rule on the basis of identified object. Finally, those sentences are used as test cases which contain a precondition, a post-condition and other related attributes. This is done for German and English languages. Sneed (2007) gave motivation for the generation of test data and expected output from natural language requirements.

Carvalho et al. (2013) propose an approach in which test cases are automatically generated from natural language requirements. These test cases are only generated for the requirements that satisfied the restriction of controlled natural language. Although, the approach in this study extracts test cases from natural language but this is only accomplished by converting requirements into Software Cost Reduction (SCR) specification that are further tested by T-VEC tool to generate vectors containing input data and expected output. However, SCR is an approach used by formal methods community which means that Carvalho et al. (2013) relied on formalizing requirements for the generation of test cases from natural language requirements. Furthermore, in Carvalho et al. (2013) no verdict assignment strategy is discussed. ReNaLART does not have any restriction to convert natural language requirements into formal requirements for verdict assignment.

2.4 | Acceptance test and behaviour-driven development tools

There are two common approaches for testing using natural language requirements. Several previous works have considered acceptance test-driven development (ATDD) and behavioural test-driven development (BDD) but none of them to the best of our knowledge has introduced a fully automated technique for test creation, and verdict assignment. ATDD is mainly used for development of agile projects. In ATDD, the acceptance criterion is defined by the users and then refined by other stakeholders. The template that is usually followed in this approach is in the form of Given, When and Then, where Given represents the precondition, When represents an action, and Then represents the post condition which means that the acceptance criterion contains input and expected outputs of the requirement to be tested. An acceptance test fails, if any acceptance criterion is not fulfilled, otherwise the corresponding test is passed. In ATDD, test definition and creation starts in the requirements phase, which helps in identifying bugs earlier, therefore, it reduces the cost of removing bugs. ATDD is also more suitable for better communication among stakeholders.

However, BDD facilitates all stakeholders to develop software through communication with each other in a language that is understandable by all stakeholders, no matter, whether these stakeholders are technical or not. In BDD, user stories and their acceptance criteria are written first.

Next a step definition file is created and finally the expected and observed outputs are compared through unit testing. What differentiates BDD from ATDD is that in BDD, an acceptance criterion is connected to an implementation by step annotations that not only supports executable behaviour but also identifies function(s) to be called.

Now we discuss some related tools for ATDD and BDD. A BDD tool called Cucumber is discussed in (Okolnychyi & Fogen, 2016) in which requirements are expressed in Gherkin form, therefore, it also supports acceptance criteria. In Cucumber, any text inside inverted commas is converted into a regular expression after running the test script. If the tester wants to match a particular string or a word in any part of Gherkin form then the tester must specify it in the form of a regular expression within associated annotation. For example, if the tester wants to extract singular of any word then it should be specified within annotation in the form of a regular expression. Similarly, digits in test data are also specified by using a regular expression. In the same way, test data and expected output are specified using regular expressions in Cucumber annotations. With each annotation, a method is associated for implementation of a scenario. In Cucumber, if any annotation is missing then it is displayed when the scenario is executed. For comparison of expected and observed outputs, usually the assert functions are used and on the basis of these test verdicts fail or pass are assigned.

In Jbehave (Okolnychyi & Fogen, 2016), requirements are expressed in Gherkin form, therefore, it also supports acceptance criteria. Moreover, test data and expected output are specified by using annotations. Like Cucumber, here also, each annotation gets associated with a method for implementing a scenario. The parameters in expected and given part of a scenario must be written after a $ sign. For comparison of expected and observed outputs, usually assert functions are used and on the basis of outcome received from them test verdicts fail and pass are assigned. The Jbehave tool, also supports unit testing.

The Concordion tool (Peterson, 2017) also supports ATDD. Requirements in it are written in an html/Markdown/ Excel file. For testing of a system, testers need to specify test data, action and expected output and these specified words are then stored in variables which are followed by a # symbol. For comparison of expected and observed outputs JUnit assert functions are used with a keyword by Concordion tool.

Spock (2018) is a BDD tool which supports ATDD. However, Gherkin form is used in Spock. However, if the requirement is not written in natural language then it does not have any effect on testing in Spock because it stores test data and expected output using some variables. It uses the equal operator for comparing the expected and observed outputs.

Easyb (Collins, 2014) is another BDD tool which supports ATDD. In Easyb test data, action and expected outputs are provided by combining the code and natural language requirements in the form of Given, When and Then. Sometimes, unit testing is used for comparison of expected and observed output in Easyb.

The assert function is used for comparison of expected and observed outputs in the Extractor tool (, 2017), which is also an ATDD tool. Fitnesse/Fit is another ATDD tool but it takes test data and expected outputs manually (Hanssen & Haugset, 2009).

2.5 | Comparison of existing related tools with ReNaLART

In this section, we compare different tools with ReNaLART (see Table 1). These tools are discussed in detail in Section 2.4. Table 1 shows the comparison of tools with ReNaLART on the basis of test data, expected output, oracle, test verdict assignment, template for expressing requirements and whether the acceptance criteria is written in Given, When, Then format or not.

ReNaLART uses a regular expression in the oracle only to detect and compare numbers in expected and observed outputs and the string distance functions are used for the comparison of strings. Therefore, ReNaLART is not dependent on any assert functions for unit testing. It also generates test data and expected output automatically from user stories without specifying any kind of annotations.

2.6 | Comparison with the state-of-the-art techniques

In this section, we compare ReNaLART with the state-of-the-art approaches available in the literature (see Tables 2 and 3). The comparison with the state-of-the-art is provided based on the requirement type, agile requirements, generation of test cases, test oracle methodology, sources for expected output generation and involvement of formal specification or models. These attributes for comparison have been identified from the literature.

In Tables 2 and 3, NL and Req have been used as abbreviations for natural language and requirement(s) respectively.

3 | RESEARCH QUESTIONS

Research in software engineering has its differences with other disciplines in terms of research question identification, types of results obtained and validating the research. We have used the criteria described in Shaw (2002) for identifying and categorizing the following research questions for this study (categories mentioned in square brackets for each research question):

TABLE 1 Tools comparison

Description	Cucumber	JBehave	Concordion	Spock	EasyB	Extractor	FitNesse/fit	ReNaLART
Test data	Semi-automatically	Semi-automatically	Manually	Manually	Manually	Manually	Manually	Automatically
Expected output	Semi-automatically	Semi-automatically	Manually	Manually	Manually	Manually	Manually	Automatically
Oracle	Assert function/ depends upon code	Assert function	JUnit assert functions with concordion keyword	Equality operator	Should/ensure syntax	Assert function	Depends upon fixture used	String distance function and regular expression
Number of test verdict	Two (pass or fail)	Two (pass or fail)	Two (pass or fail)	Two (pass or fail)	Two (pass or fail)	Two (pass or fail)	Two (pass or fail)	Four (pass, fail, warning, inconclusive)
Template/ requirement	Gherkin form	Gherkin form	HTML/ markdown/ excel file	May or may not support user story	Story format	Plain text	Table	User story
Acceptance criteria format	Yes	Yes	Yes/no	Yes/no	Yes	No	No	No

TABLE 2 Comparison of ReNaLART with other natural language-based testing approaches

Papers	Restricted Natrual Language/Natural Language Requirements	Formal specification	Agile Requirements	Test cases	Models involved	Test oracle using assertion/ JUnit	Test oracle (other)	Test oracle for Natural Language
Zhang et al. (2014)	✓			✓	✓			
Yue et al. (2015)	✓			✓	✓	✓		
Carrera et al. (2014)	✓		✓	✓	✓	✓		
Sarmiento et al. (2014)	✓			✓	✓			
Aichernig et al. (2014)	✓	✓		✓	✓		✓	
Wang et al. (2015)	✓	✓		✓	✓			
Wang et al. (2017)	✓	✓		✓	✓		✓	
Elghondakly et al. (2015)	✓		✓	✓				
Rane (2017)	✓		✓	✓	✓			
Sneed (2007)	✓			✓				
Carvalho et al. (2013)	✓	✓		✓				
ReNaLART	✓		✓	✓				✓

Abbreviations: NL, natural language; Req, requirement(s); ReNaLART, Restricted Natural Language Agile Requirements Testing.

RQ1: Which format or template will be appropriate for writing natural language requirements in an agile context to automate the testing process? [Method or means of development].

RQ2: How to generate and execute test data automatically from the requirements template, chosen in response to RQ1? [Method or means of development].

RQ3: How to extract expected output(s) for requirements specified in the chosen template, in response to RQ1? [Method or means of development].

RQ4: How to automate the test oracle by comparing observed and expected outputs, received in RQ2 and RQ3? [Method or means of development].

RQ5: How effective is it to test the software in an agile context from natural language test oracle? [Method for Analysis\ Evaluation].

TABLE 3 Comparison of ReNaLART with other natural language test oracles

Papers	Sources for expected output generation	Test oracle (other)	Test oracle for Natural Language
Goffi et al. (2016)	Exception comments from Java doc	✓	
Mayer and Guderlei (2004)	Generation of expected output using random inputs	✓	
Hoffman (1998)	Generation of outputs using heuristic approach	✓	
Hoffman (1998)	Generation of outputs using stochastic approach	✓	
Hoffman (1998)	Generation of outputs using true oracle		
Hoffman (1998)	Generation of outputs using sampling approach	✓	
Hoffman (1998)	Generation of outputs using previous version	✓	
Peters and Parnas (1998)	Program description	✓	
Lucca et al. (2002)	Decision table	✓	
Memon et al. (2000)	Formal model and test case	✓	
Last et al. (2004)	IFN model	✓	
Manolache and Kourie (2001)	Model program	✓	
ReNaLART	Restricted user story		✓

Abbreviations: IFN, Info Fuzzy Network; NL, natural language; ReNaLART, Restricted Natural Language Agile Requirements Testing.

4 | ReNaLART

ReNaLART proposes an automated oracle that compares the expected and observed outputs and then assigns test verdicts depending on this comparison. As the agile requirements are written in the form of user stories in natural language, therefore, an oracle that can give verdicts by processing natural language requirements is needed. The ReNaLART oracle is further strengthened by instrumenting it with multiple string matching algorithms for processing of natural language strings in the observed and expected outputs.

Figure 1 depicts the ReNaLART testing framework and shows how a user story is split into 'As a', 'I want', 'so that' and 'shall' or 'will' or 'should'. The test data and expected output are generated from their respective representations. Stop words are then removed from both the observed and expected outputs. Finally a verdict is assigned using the regex pattern and string distance functions. The regex pattern is used for the comparison of digits in the expected and observed outputs while the remaining string is compared using string distance functions. The next section provides a detailed discussion of ReNaLART.

4.1 | RQ1: Format of agile requirements in natural language

The agile requirements are written in the form of user stories in projects developed following the agile methodology (Lucassen, Dalpiaz, van der Werf, & Brinkkemper, 2017). A discussion about a suitable template is provided in the next section.

4.1.1 | User stories

There are different templates available to write user stories. We propose to write user stories in the Connextra template (Role-feature-reason, 2017) which is also used in (Lucassen et al., 2017). Connextra is also recognized as the most widely used template for expressing user stories in Lucassen and Dalpiaz (2016). This template has the following format:

> As a (type of user), I want (some goal) [so that (some reason)].

We extend this template by adding some other natural language restrictions for the extraction of test data and expected output as discussed in Sections 4.2 and 4.3. In ATDD, the criteria is written in the Given–When–Then form while in ReNaLART there is no need to mention acceptance criteria along with each scenario separately, the user stories themselves provide the acceptance criteria.

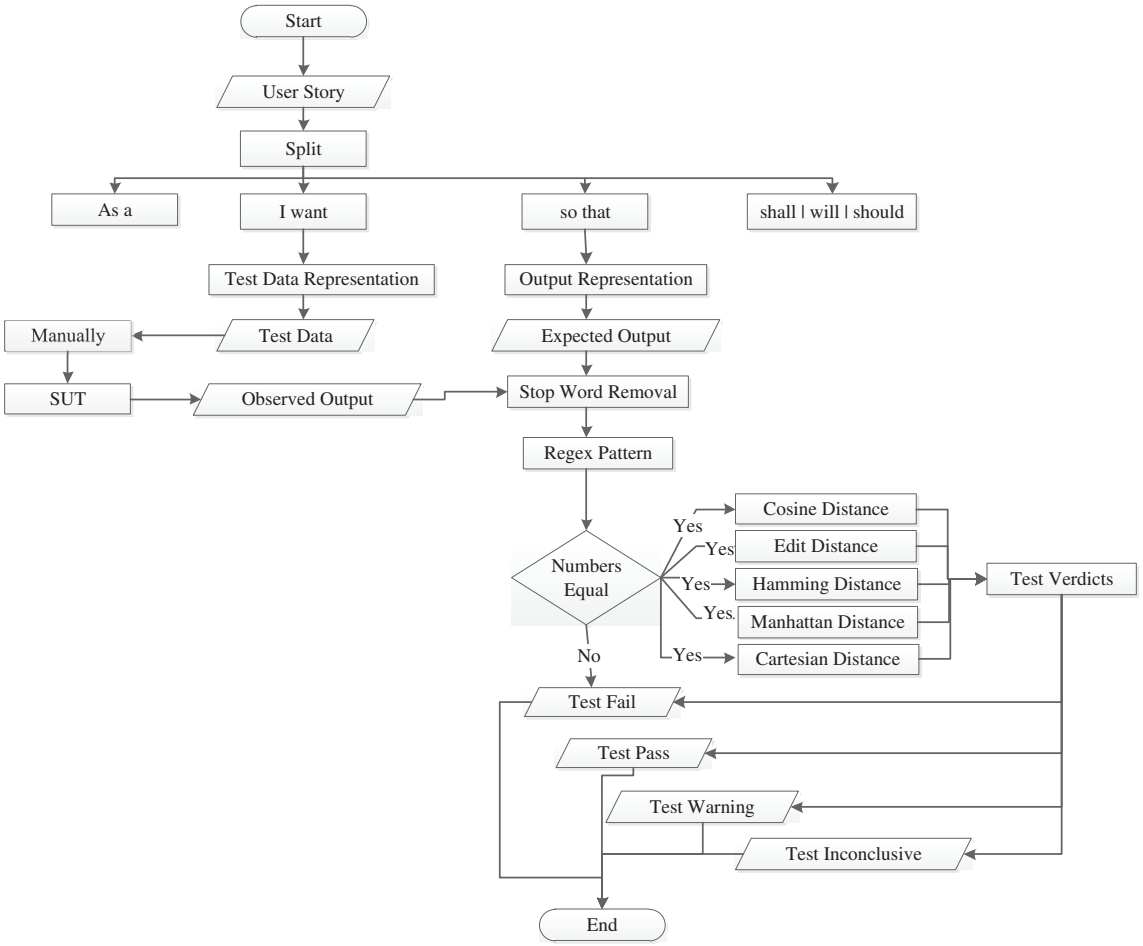

FIGURE 1 The ReNaLART test oracle framework. ReNaLART, Restricted Natural Language Agile Requirements Testing

4.1.2 | Split

An efficient automated test oracle strategy based on agile requirements should be able to extract expected output from the user stories. The template described in Section 4.1.1 for describing user stories along with some other restrictions can suitably be used for extraction of expected output from natural language agile requirements. In ReNaLART, user stories are split by the following words, as also discussed in Robeer, Lucassen, van der Werf, Dalpiaz, and Brinkkemper (2016):

- 'As a'.
- 'I want'.
- 'So that'.
- 'Will' or 'shall' or 'should'.

4.2 | RQ2: Automatic test data generation

The user stories are written in restricted natural language which also contains keywords. We propose an approach to generate test data from a user story by taking the text between 'I want to' and 'so that' part of a user story. The regular expression of Equation (1) provides that part of the user story from which test data are extracted.

$$\text{^As a}(.*)\text{I want}(.*)\left['\left[\boxed{(.*)}(,)?\right]+'\right](\text{and}(\text{click on}|\text{run}|\text{execute})(.*))?\$ \quad (1)$$

The test data are extracted from within the single quotes which is represented by a box in the regular expression of (1). The user can represent the input data within the single quotes in two ways. In the first case, the test data are generated directly from within the single quotes. For example, *test data*: 'Crispy chicken' is automatically extracted from the following User Story:

> As a user, I want to place an order by selecting 'Crispy chicken' and click on add to my order so that 'shopping cart' will be displayed.

In the second case, the user can use an equal operator within the single quotes. This helps in scenarios where the user needs the test data corresponding to each label of a widget. In such a situation, the words before equal represent some label which is stored separately and test data are generated from the user story after '=' automatically. For example, *test data*: 'Miss, XYZ, Female, xyz@xy.z.uv, 123456789, 123456789' is automatically extracted from the following User Story:

> As a user, I want to register an app by entering the following personal information: 'Salutation = Miss, First Name = XYZ, Gender = Female, Email address = xyz@xy.z.uv, Password = 123456789, Re-enter Password = 123456789' and click on Next so that 'Terms and Condition' will be displayed.

4.3 | RQ3: Automatic expected output extraction

To extract expected output from the user story ReNaLART takes the text between 'so that' and 'will' or 'shall' or 'should' part of the user story. The expected output is extracted from the user story after applying the split function on the user story and converting the text into lower case. The regular expression (2) provides that part of the user story from which expected output is generated.

$$\text{^so that}(.+)\left['\boxed{(.*)}(,)?'\right]\ \text{will}\,|\,\text{shall}(.*)\$ \tag{2}$$

The expected output is generated within the single quotes in the regular expression and is represented by a box in regular expression (2). If there are multiple outputs of a single user story, then these are separated by commas and extracted automatically from the user story.

For example, *expected output*: 'Targeted Feedback' is generated after applying ReNaLART on the following User Story:

> As a user, I want to answer question 1 of pre-course assessment by selecting 'C' and clicking on Check Answers so that 'Targeted Feedback' will be displayed.

4.4 | Stop word removal

ReNaLART removes stop words to make the comparison of expected and observed outputs robust. Stop words are those words that vary from one domain to the other. In our approach, stop words are those words which are not significant and discarding them does not influence determining the correct behaviour of the SUT. Therefore, such words are removed from expected and observed outputs. For this purpose, we have used the Stanford Core NLP stop word list available at (Stanford, 2017). Some words in the Stanford stop word list are essential for checking the correct behaviour of the SUT. For example, the words which are used to express a negation are important for comparison of expected and observed outputs. Therefore, for the purpose of software testing we have not included all words from the Stanford list in the stop word list.

For example, if there is an expected output 'Not Edited Successfully' and the observed output obtained after running the SUT is 'Edited Successfully'. Then in this case, if the word 'not' is considered as a stop word and removed from the expected output then the expected and observed outputs will be the same, which is not correct. Therefore, we have not included all the words in the stop words list and have chosen the words which are most relevant to software testing.

Consequently, in the *Stop Word Removal* module of ReNaLART, all words of the expected output are tokenized for comparison with the words in the stop word list. If any word from the tokenized expected output matches with any word in the stop word list, then, that respective word is removed from the expected output. Similarly, stop words are also removed from the observed output.

4.5 | RQ4: Test verdict assignment

The automated oracle for comparison of output in ReNaLART works in two parts: First, for each user story all digits in the expected and observed outputs are extracted by using the regex pattern. The oracle then compares digits of expected and observed outputs with the same indices. If all digits in the expected and observed outputs are the same then the second step of the oracle is proceeded.

The first part also compares the values before decimal part for floating point numbers. The rest of the values after decimal numbers are ignored. Consequently, the results are not affected even when the digits after the decimal are slightly different due to different reasons like the variations in the underlying hardware architecture, operating system or development programming language.

However, if the digits in the expected and observed outputs are not the same then, the oracle does not proceed to second step and a fail verdict is assigned.

For example, for a user story having the expected output: 'Pages 250', the observed output obtained is: 'Pages 200'. Since the number of pages in this case are not the same in both the outputs, therefore, a fail test verdict is assigned and there is no need to apply the string distance functions.

In the second step, the automated oracle compares the words/characters in the expected and observed outputs using the string distance functions and assigns test verdicts based on these comparisons.

In the proposed approach, the expected and observed outputs are converted into lower case to avoid ambiguities in test verdicts with same words in a different alphabetical case in the observed and expected outputs.

For comparison of expected and observed outputs and assigning test verdicts, we have used five string distance functions which are: Cosine, Edit, Hamming, Manhattan and Cartesian. These string distance functions are selected because they have already been used in software testing field for string test case generation in (Shahbazi & Miller, 2016). Similarly, Edit, Hamming, Manhattan and Cartesian distance formulas have also been used in (Ledru, Petrenko, Boroday, & Mandran, 2012) for test case prioritization. In ReNaLART these string distance functions are applied on expected and observed outputs of each user story to assign test verdicts.

We have used three different test verdicts with each string distance function which include: a pass, a fail and a warning. For all string distance functions, the test pass verdict is assigned when the expected and observed output strings are exactly the same. The warning test verdict is assigned owing to minor difference(s) in the compared outputs. And the fail verdict is assigned if the outputs are totally different or major difference(s) within the compared outputs exist. We have identified the thresholds for major and minor differences for each string distance function using experiments and have used these for the test verdict assignment.

The test pass verdict for all string distance functions is assigned using the zero distance value between the expected and observed outputs. For the Edit, Hamming and Cosine distance functions the warning test verdict is assigned when 80% of characters/words are the same in the observed and expected outputs. This criterion represents a minor change within the compared outputs. We have experimentally identified the percentage of correct characters for Edit and Hamming distance functions to set the threshold values. From our experiments we found that a similarity of greater than 80% and less than 100% is a reasonable threshold for assigning a warning verdict using the Cosine distance function. However, it is not possible to set the 80% criteria for the Manhattan and Cartesian distance functions. Therefore, in their case a warning verdict is assigned when change(s) are detected towards the end of the output. If the test is not a pass or a warning, then it is a fail for all string distance functions.

ReNaLART also computes an aggregate verdict by combining the results of all five string distance functions. The four different aggregate verdicts that can be assigned to test cases include: a pass, a fail, a warning and an inconclusive test verdict. In the following sections, we describe the string distance functions which ReNaLART uses and the criteria for assigning verdicts with respect to each string distance function.

4.5.1 | Cosine distance

The Cosine distance measures distance between two vectors. If the vectors are exactly the same then their distance is 0. On the other hand, if they are orthogonal, their cosine distance is 1. We have used the cosine distance on expected and observed outputs. Two vectors are generated by counting each word in the expected output and observed outputs such that if any word occurs more than once in the expected or observed outputs then the value is incremented by one in the corresponding output vector. The Cosine distance has been computed as in (Shahbazi & Miller, 2016) by applying the following formula:

$$\text{Cosine Similarity}\ (\text{obs},\ \text{exp}) = \frac{\sum_{i=1}^{n} \text{obs}_i . \text{exp}_i}{\sqrt{\sum_{i=1}^{n} \text{obs}_i^2} \cdot \sqrt{\sum_{i=1}^{n} \text{exp}_i^2}}, \tag{3}$$

$$\text{Cosine Distance}\ (\text{obs},\ \text{exp}) = 1 - \text{Cosine Similarity}\ (\text{obs},\ \text{exp}), \tag{4}$$

where obs_i and exp_i are the term frequencies of one word in the expected and observed outputs respectively. $obs_i \cdot exp_i$ is the dot product of expected and observed output vectors which is divided by the magnitude of expected and observed output vectors. For example, if the expected output is 'Results for Unit 1 Introduction' and observed output is 'No results for Unit 1 Introduction' then a cosine similarity of 0.91 is obtained using Equation (3) and a distance of 0.09 is obtained using Equation (4). The result of cosine distance close to 0 as in this case depicts that fewer words (one word in this case) are different in both outputs.

Table 4 shows the criteria for assigning test verdicts. If all the words in the observed and expected outputs are the same, then, a pass test verdict is assigned. The range for assigning a warning test verdict is determined after some experimentation with different values. Our experimentation showed that a range of more than or equal to 80% and less than 100% of the total words matching in the expected and observed outputs to be appropriate for assigning a warning verdict. The ranges mentioned in Table 4 to assign different verdicts have been obtained through experiments and analysis of results of these experiments. For example, if eight words are the same in both the outputs and only two words are different then in such a case a warning test verdict is assigned. On the other hand, if the result is even not within the range of a test warning then a fail test verdict is assigned. As an example, after applying the Cosine distance on the following User Story having the same expected and observed outputs, a pass verdict is assigned:

> As a user, I want to answer question 4 of post-course assessment by selecting 'Olympic Games' and click on Check Answers so that 'Your score is: 0/5' will be displayed.

4.5.2 | Edit distance

The second formula used is called the Edit distance or Levenshtein distance. The Edit distance is a string distance function which measures the dissimilarity between two strings. It supports three operations: insertion, deletion and substitution. Each operation has a cost of one associated with it. Moreover, in Edit distance the minimum application of these operations is desirable for making two strings similar to each other. We computed it as in Navarro (1998, 2001), Shahbazi and Miller (2016), Soukoreff and MacKenzie (2001) by applying the following formula:

$$\begin{aligned}
&\text{Initially, } \operatorname{edit}(obs_i, 0) = obs_i \\
&\qquad \operatorname{edit}(0, exp_j) = exp_j \\
&\operatorname{edit}(obs_i, exp_j) = \min \begin{cases} \operatorname{edit}(obs_{i-1}, exp_j) + 1 \\ \operatorname{edit}(obs_i, exp_{j-1}) + 1 \\ \operatorname{edit}(obs_{i-1}, exp_{j-1}) + \operatorname{cost}(obs_i, exp_j) \end{cases} \\
&\operatorname{cost}(obs_i, exp_j) = \begin{cases} 0 & obs_i = exp_j \\ 1 & \text{otherwise} \end{cases}
\end{aligned} \tag{5}$$

where exp and obs represent the observed and expected outputs, and obs_i and exp_j are the locations of characters in the corresponding outputs. In this case, if two characters are the same then it has a cost of 0 but if any insertion, deletion or update operation is required then it has a cost of 1.

Table 5 shows the criteria for assigning test verdicts. If the Edit distance results in a value equal to zero then it means that observed and expected outputs are same therefore, a pass test verdict is assigned. The percentage of corrected characters used for a warning test verdict in Table 5 is obtained by subtracting the edit distance from the total number of characters in the longer output, then dividing it by the total number

TABLE 4 Test verdict criteria for Cosine distance

Test verdict	Reason
Test pass	Distance == 0.0
Test warning	Distance > = 0.01 && distance <= 0.20
Test fail	For all other values

TABLE 5 Test verdict criteria for Edit and Hamming distance

Test verdict	Reason
Test pass	Distance == 0.0
Test warning	Correct characters percentage > = 80% && correct characters percentage < 100%
Test fail	For all other values

of characters in the longer output string and then multiplying it by 100. Furthermore, Table 5 shows that a fail test verdict is assigned in case the result does not meet the criteria of a test warning or a test pass.

For example, a distance of 0.0 is obtained after applying the edit distance on the user story discussed earlier resulting in a pass test verdict in this case also.

4.5.3 | Hamming distance

Hamming distance is applied only on equal length strings and it supports only one operation, that is, substitution (Burkhardt & Kärkkäinen, 2001). However, we have also computed the Hamming distance for unequal length strings using the following formula as in (Cooper, 2016; Frakes & Fox, 2003; Ledru et al., 2012; Shahbazi & Miller, 2016):

$$(\text{longest} - \text{shortest}) + \text{Hamming distance}\ (\text{obs}[1...l1],\ \text{exp}[1...l2]) \tag{6}$$

where obs and exp are the observed and expected outputs having lengths $l1$ and $l2$, respectively. In the formula (longest – shortest) is used for calculation of distance if the lengths of expected and observed outputs are not equal. In (6), the longest represents the longer length output and the shortest represents the shorter length output.

Table 5, shows the criteria for assigning pass, warning and fail test verdicts. The percentage of corrected characters in Table 5, is obtained by subtracting the Hamming distance from the total number of characters in the longer output then dividing it by the total number of characters in the longer output and finally multiplying it by 100.

As an example, after applying the Hamming distance on the user story discussed earlier, a distance of 0.0 is obtained, therefore, a pass test verdict is assigned in this case also.

4.5.4 | Manhattan distance

Manhattan distance is computed using the following formula as suggested in (Cha, 2007; Ledru et al., 2012; Leskovec, Rajaraman, & Ullman, 2014; Shahbazi & Miller, 2016):

$$\text{Manhattan distance} = \sum_{i=1}^{n} |\ \text{exp}_i - \text{obs}_i\ | \tag{7}$$

where exp_i and obs_i are characters of the expected and observed output strings. If the expected and observed outputs are not equal in length then null characters are padded to the shorter string to make its length equal to the longer string. For example, if the expected output is 'job title bank manager' and the observed output is 'job title manager' then a distance of 642 is obtained after applying the Manhattan distance function (strings lengths are made equal first by using padding). In this case a comparison between each character of both outputs at the same index is done and if these characters are different then a difference of their ASCII values is computed. Then the differences for all the respective character locations in both the strings are aggregated.

Table 6 shows the criteria for assigning test pass, test fail and test warning verdicts. In Table 6, 122 represents the greatest ASCII value for English characters ranging from *a* to *z*. We have observed that if there is some character change by insertion or deletion of a character towards the end of the output strings then it lies within the mentioned range in Table 6 for a warning test verdict. Furthermore, a warning verdict in this case is illustrated by character insertion or deletion towards the end of output. It is quite possible that there is a character change due to substitution, in such a situation a test verdict assignment depends upon the distance value computed.

As an example, after applying the Manhattan distance function on the user story discussed earlier, a distance of 0.0 is obtained, therefore, a pass test verdict is assigned in this case.

TABLE 6 Test verdict criteria for Manhattan and Cartesian distance

Test verdict	Reason
Test pass	Distance == 0.0
Test warning	Distance <= 122 + length of largest output
Test fail	For all other values

4.5.5 | Cartesian distance

The Cartesian distance is also called the Euclidean distance which is the distance between two points, characters or terms. In ReNaLART it is computed using the following formula as suggested in (Huang, 2008; Ledru et al., 2012; Leskovec et al., 2014; Shahbazi & Miller, 2016):

$$\text{Cartesian distance} = \sqrt{\sum_{i=1}^{n}(\text{exp}_i - \text{obs}_i)^2} \tag{8}$$

where exp_i and obs_i are the characters of expected and observed output strings at the ith location. If the expected and observed outputs are not equal in length then null characters are padded to the shorter string to make it equal in length to the longer string. For example, if expected output is 'job title bank manager' and observed output is 'job title manager' then a distance of 246.84 is obtained after applying the Cartesian distance function (padding is used in this case also). In this case a comparison between each character of both outputs at the same index is done and if these characters are different then a difference of their `ASCII values` is computed. The differences for all characters in the strings are aggregated and then the square root of the aggregated difference is computed to get the final result.

Table 6 shows the criteria for assigning test pass, fail and warning verdicts using the Cartesian distance function. It shows that if the result of Cartesian distance formula is equal to zero then the observed and expected output strings are the same, therefore, a pass test verdict is assigned. A warning test verdict is assigned if the Cartesian distance is less than or equal to 122 plus the length of longer output string, otherwise, a fail verdict is assigned which is the case when the condition for a pass or a warning test verdict does not hold.

4.5.6 | Final test verdict

Finally, after applying the above described five string distance functions, the next step is to assign an aggregate test verdict as a pass, a fail, a warning or an inconclusive. We have proposed a decision tree (see Figure 2) to assign a single aggregate test verdict for each respective test case.

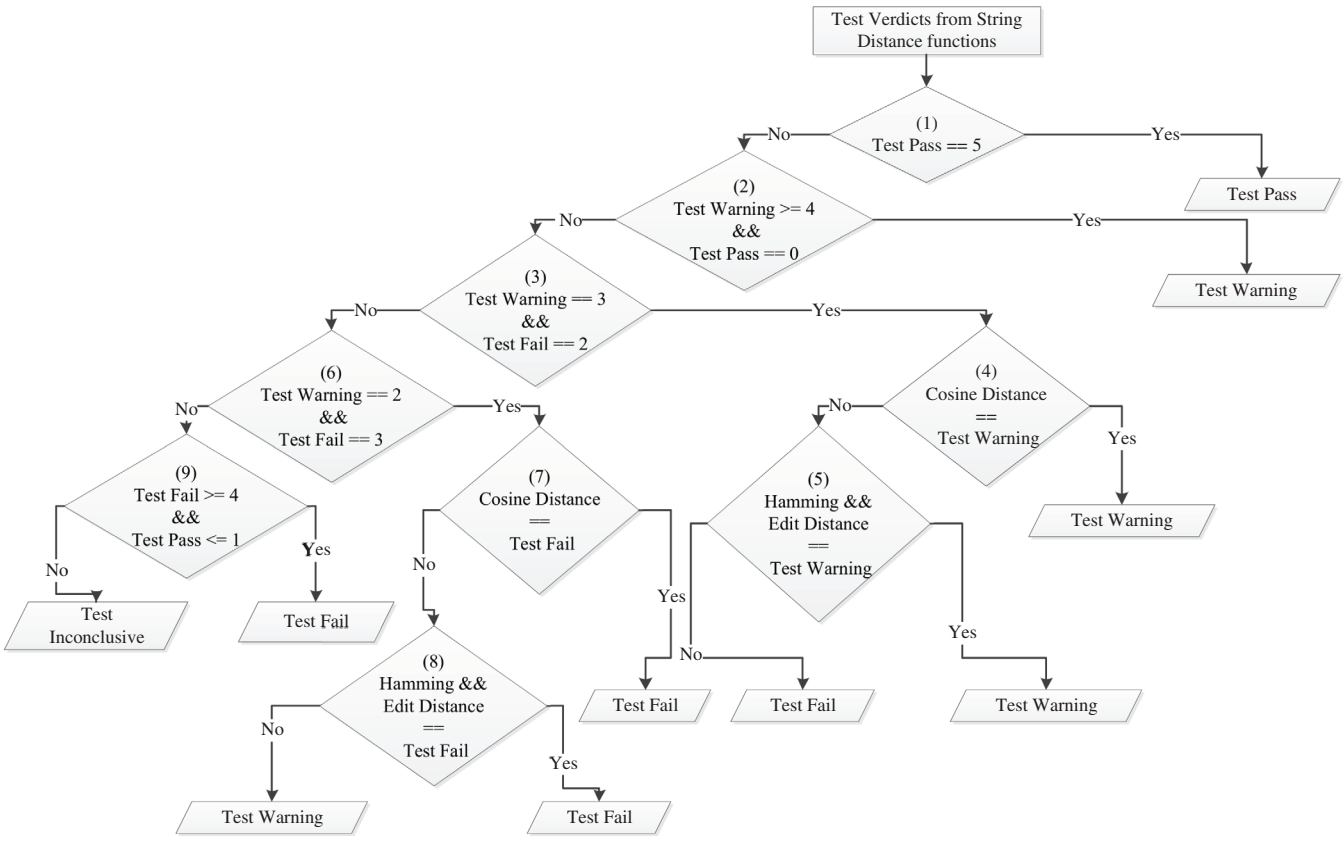

FIGURE 2 Decision of test verdicts

The notion behind this decision tree is to aggregate all possible combinations of test verdicts arising from string distance functions into a single aggregate verdict for each test case. The final test verdict is a pass if we get all five pass verdicts.

Moreover, ReNaLART assigns warning, fail or inconclusive final verdicts on the basis of prioritization of string distance functions on the basis of the decision tree shown in Figure 2.

For prioritization of the string distance function it has been observed that changes in words have more significance as compared to characters, therefore, the first priority is given to the Cosine distance which compares the expected and observed outputs at word level while Edit, Hamming, Manhattan and Cartesian distances perform comparisons at character level in ReNaLART. The second priority is given to Edit and Hamming distance functions. Edit and Hamming distances are given second priority because for these distance functions we can quantify the character changes for warning verdicts from greater than equal to 80% but for Manhattan and Cartesian distances such quantification is not possible for character changes due to substitution. However, in this case a warning verdict can be given on the basis of character changes by insertion and deletion towards the end of the output. Therefore, owing to this weak warning verdict assignment in Manhattan and Cartesian distances, they have been given the third priority which is also strengthened by (Huang, 2008) in which Cartesian distance was considered to be the worst function in terms of performance for clustering of text documents. Furthermore, this prioritization of string distance is only applied when one of the following condition meets as also shown in Figure 2.

- Test Warning == 3 and Test Fail == 2.
- Test Fail == 3 and Test Warning == 2.

In the above conditions, test warning == 3 means three verdicts are assigned as a warning after applying the string distance functions and Test Fail == 2 means two fail verdicts are assigned after applying these string distance functions.

If any of the above condition is true (see decision Boxes 3 and 6 in Figure 2), then the verdict assigned from the Cosine distance function is checked (see Boxes 4 and 7 in Figure 2). The final verdict is a warning or a fail depending upon the verdict given by the Cosine distance function (see 'Yes' branches of boxes 4 and 7). Otherwise, the 'No' branch of decision Boxes 4 and 7 will be executed to assign aggregate verdict by means of second priority distance functions, that is, Hamming and Edit distance functions.

ReNaLART also deals with a scenario in which possibly a conflicting combination of test fail, test pass and test warning verdicts arise from different string distance functions. In such a case, ReNaLART assigns a test inconclusive verdict. The inconclusive verdict helps to deal with the conflicting results. Even though this situation arose rarely in our experiments owing to an otherwise stringent verdict assignment mechanism but still we have proposed this verdict to address the scenario of conflicting results mentioned above. For a warning or a fail verdict, expected and observed outputs are compared, if they have equal lengths then words absent in the observed output are extracted for the user.

For example, after applying the string distance functions on the User Story mentioned in Section 4.5 all the string distance functions return a pass verdict, therefore, an overall pass test verdict is assigned.

5 | EXAMPLE

Now we illustrate ReNaLART by using the following user stories.

> User Story: As a user, I want to wait for 2 seconds for HTML quiz after selecting answer of question 1 by 'Hypertext Markup Language' so that 'Time spent 0:02' will be displayed.

After applying ReNaLART on the above user story test data 'Hypertext Markup Language' and expected output 'Time spent 0:02' are generated. The observed output corresponding to this user story is 'Time spent 0:00'. ReNaLART, first extracts all digits in the expected and observed outputs by using the regex pattern as the numbers are different, therefore, ReNaLART will not move to the second step for comparison of expected and observed outputs to assign test verdicts by using five different string distance functions. A test fail verdict is assigned in this case.

Let us take another user story through which we can elaborate other possibilities of verdict assignment using ReNaLART.

> User Story: As a user, I want to clear location filter by setting 'Accept' so that 'Current location' will be displayed.

Applying ReNaLART on the above user story generates test data 'Accept' and expected output 'Current location'. The observed output corresponding to this user story is some other location which is preset as 'XYZ'. ReNaLART, first checks the digits in the expected and observed outputs by using the regex pattern as there are no numbers therefore, it will move to the second step for comparison of expected and observed

outputs to assign test verdicts by using five different string distance functions. All the string distance functions assign a fail verdict. Therefore, by using the decision tree of Figure 2, an overall fail test verdict is assigned.

Let us take another user story through which we can elaborate ReNaLART further.

> User Story: As a user, I want to search by 'Enter a search term = Unit 1 – Introduction' and click on Search so that 'Results for Unit 1 Introduction' will be displayed.

Application of ReNaLART on the above user story gives test data as 'Unit 1 – Introduction' and expected output as 'Results for Unit 1 Introduction'. If we get 'No results for Unit 1 Introduction' as the observed output. ReNaLART first extracts all digits in the expected and observed outputs by using the regex pattern which gives the same numbers, therefore, it moves to the second step for comparison of expected and observed outputs to assign test verdicts by using five different string distance functions. In this case, Hamming distance, Manhattan distance, Cartesian distance assign a fail verdict while Cosine distance and Edit distance assign a warning verdict. By using the decision tree, overall a warning test verdict is assigned.

Let us take another user story through which we can elaborate on the test inconclusive verdict.

> User Story: As an employee, I want to search 'Marketing Assistant name' so that 'Satomi Hayakawa' will be displayed.

ReNaLART generates Marketing Assistant Name and Satomi Hayakawa as the test data and expected output, respectively. The generated observed output has the last name followed by the first name as Hayakawa Satomi. The regex pattern does not identify any number due to the absence of digits in the outputs. Therefore, ReNaLART moves to the next step to generate test verdicts using string distance functions. ReNaLART assigns a test pass verdict using the Cosine distance function. Furthermore, it assigns three fail and a warning verdict using the remaining distance functions. Therefore, in this case an overall inconclusive test verdict is assigned.

6 | EXPERIMENTAL SET-UP

We developed a working prototype of ReNaLART in the Java language using the Netbeans IDE. The experiments were performed on a Dell PC having Intel(R) Core(TM) i7-8550 CPU, a RAM of 8 GB and having Windows 10 operating system.

We applied ReNaLART on several case studies from different domains as shown in Table 7 which shows the average ratings and the number of downloads for the android apps tested with our approach. These data were, however, not available for the web applications tested with ReNaLART. Now we provide a brief description of these applications in the following paragraphs.

6.1 | OLX Pakistan

OLX Pakistan (2019) is an Android shopping app. The app contains ads for both new and used products. The app facilitates users to buy products from multiple categories including mobile, books, car, houses, home appliances, business, industry and so on. It also provides a diverse searching feature for any product. The users can also sell their products by providing the relevant details against the product to be sold. Moreover, the app also provides the facility to add a job or search a relevant job within a specified location or field.

TABLE 7 Case studies

Case study	Domain	Rating	Downloads
OLX Pakistan	Shopping android app	4.6	5 M+
Mental health tests	Medical android app	4.0	50 k+
McDelivery Pakistan	Food & drink android app	3.7	500 k+
BlueStacks	Desktop app	N/A	300 M+
Power Searching with Google	Educational website	N/A	N/A
TensorFlow playground	Artificial intelligence website	N/A	N/A
w3Schools 2018 offline	Educational android app	4.1	100 k+
Touch'D	LifeStyle android app	4.1	5 k+

6.2 | Mental Health Tests

Mental Health Tests (2019) is an Android medical app which provides the facility to the user to test their mental health. The app contains multiple categories of mental health tests including depression, anxiety, adult attention-deficit/hyperactivity disorder (ADHD) and so on. The app provides results against each test along with the recommendation.

6.3 | McDelivery Pakistan

McDelivery Pakistan (2019) is an Android app in which the services are provided by McDonalds within Pakistan.

6.4 | BlueStacks

BlueStacks (2019) is a desktop application which provides the facilities to access Android apps on the computer. The apps are downloadable from Google Play Store.

6.5 | Power Searching with Google

The Power Searching with Google (2018) is one of the search websites that is developed using course builder. The Course Builder (2018) provides online courses. It also provides the facility for course creation. In Power Searching along with online course content, test assessment is also provided. A test is taken before and after covering the course content for accessing the information gained by undertaking a course. Moreover, the test assessments are also conducted after covering each unit.

6.6 | TensorFlow playground

The TensorFlow Playground (2018) case study is an open source Google project that generates the results for the neural network. The results are shown both visually and using natural language. The neural network works accordingly to the input given by the user. The user can provide several input values containing learning rate, regularization, activation, problem type, and regularization rate. The output is shown using natural language in the form of test loss and training loss.

6.7 | w3Schools 2018 offline

The w3Schools 2018 offline (2019) is an Android education app which provides the facility to the user to learn about multiple web technologies.

6.8 | Touch'D case study

Touch'D (2017) is another Android app that serves as an interaction medium between family and friends. By using Touch'D the user not only connects via phone with their friends and family but also connects via WhatsApp, Gmail, Facebook and Twitter. However, sustaining records of interaction type and interaction day only depends upon user preferences of choosing a media. Touch'D maintains the relationship health of a user with each contact in the contact list. The relationship health is a measure of bonding among users with their family and friends. Usually, close persons are added as favourites and if for some reason a user cannot contact any favourite for a certain time then a notification is generated that indicates the time duration since the last interaction and also recommends to send a message or make a call. Touch'D users have their profile that contains personal information, education, working information and so on. Some of the information like date of birth and education is extracted from the user account. Touch'D users can edit some information on their profile, update their mood and status as desired and also set privacy settings. Furthermore, if there are public events or events like birthdays then it is not required to set reminders for wishing. Touch'D provides a facility to automatically generate reminders about special events for wishing to friends and family. Another facility available in Touch'D is the setting of notes for each contact which are displayed whenever there is a voice call interaction between the user and the person associated with the note. Touch'D users also get updates about the activities of their friends on Facebook and Twitter.

7 | RESULT ANALYSIS AND DISCUSSION

In order to address RQ5 we evaluate the effectiveness of the proposed approach in an agile context using several case studies of different domains. In Segura, Fraser, Sanchez, and Ruiz-Cortés (2016), authors measure the effectiveness of a testing technique in general and metamorphic testing in particular (because it has no counterparts for giving verdicts in the absence of a test oracle) by its ability to find 'unreported' faults in 'real' software. They categorize unreported faults to be those faults which have not been reported earlier and real software to be that software which is available in the public domain. Like metamorphic testing which has no counterparts to give verdicts in the absence of a test oracle, ReNaLART to the best of our knowledge also does not have any counterpart which gives verdicts on the basis of natural language-based test oracle. Therefore, we have evaluated the effectiveness of ReNaLART on the basis of the aforementioned two factors. The results shown in Table 8 show the statistics of faults revealed by ReNaLART which have not been reported elsewhere in the literature to the best of our knowledge. Plus, the software shown in Table 7 and tested with ReNaLART is available in the public domain as free Android apps or freely accessible websites. Most of the apps have a lot of downloads as evident from the statistics in the table. We generated, test data and expected output for each user story of the mentioned case studies. Furthermore, ReNaLART assigned test verdicts after comparing the observed and expected outputs of each user story.

First we discuss the results of Power Searching with Google case study in which ReNaLART revealed several errors. A total of 11 requirements were tested, out of which 5 requirements passed the relevant tests, 1 resulted in a warning verdict and 5 requirements were assigned fail test verdicts. For example, a fail test verdict corresponding to a user story test input was assigned because its execution on the SUT revealed that the page did not not have sufficient information regarding the forum and had no title. It only contained the 'help' link which further did not contain sufficient information. ReNaLART also revealed an error related to the search field producing no results for Unit 1.

Another error concerned the displaying of targeted feedback with the provided answer. The SUT requirement in this case mentioned to provide the facility of providing the suggestions with each assessment. However, this requirement was not fulfilled by the SUT and was successfully revealed as an error. Furthermore, activity 6.2 of the SUT (Power Searching with Google) failed to provide correct answers for some inputs. For example, if the user enters Chinese sushi as a test input then it should generate an alert for wrong choice because the hint suggests that the answer should be marked correct for Japanese sushi only. However, the SUT returns the correct answer for Chinese sushi also. An analysis of the SUT responses revealed that it returned the answer to be correct no matter if the user input contained either the word Japanese or sushi in it. ReNaLART successfully revealed this erroneous behaviour. All these identified faults were validated from the Course Builder Team at Google responsible for development of Power Searching with Google and they confirmed these faults to have correctly been identified by ReNaLART.

ReNaLART also successfully identified faults in OLX Pakistan App. It assigned fail test verdicts for some user stories related to clearing filters because the SUT did not clear all the fields. ReNaLART assigned a fail test verdict in an interesting scenario for a user story in which the category is semantically similar but syntactically different. This shows that currently, ReNaLART works only for syntactic similarity of observed and expected outputs.

The BlueStacks desktop application was also tested by the execution of two android apps: 'Mental Health Tests' and 'McDelivery Pakistan'. To validate ReNaLART in this case, same user stories for both the case studies were used. Our findings show that the BlueStacks application does not display some characters or digits on the screen for some user stories of Mental Health Tests App, therefore ReNaLART assigned test fail verdicts for the respective user stories.

W3CSchool App does not provide the pages against several links which was successfully identified by ReNaLART. Further, the time spent on starting the quiz was also not incremented which has been revealed as an error. And for some quizzes Question 1 was repeated twice which has also revealed as an error by ReNaLART.

Several faults were also identified in Touch'D case study. In some Touch'D user stories, the label and button were changed due to which expected and observed outputs were different and a fail test verdict was assigned. For some user stories, semantically similar but syntactically

TABLE 8 Results

Case study	Total requirements tested	Test pass	Test warning	Test inconclusive	Test fail
Power Searching with Google	11	5	1	0	5
OLX Pakistan	18	15	0	0	3
Mental Health Tests	17	17	0	0	0
McDelivery Pakistan	6	6	0	0	0
BlueStacks	23	19	0	0	4
TensorFlow playground	8	8	0	0	0
w3Schools 2018 offline	43	17	0	0	26
Touch'D	23	8	1	0	14

TABLE 9 Average test oracle time in sec for all case studies in ReNaLART

Case study	Test data and expected output generation	Verdict assign	Total
OLX Pakistan	0.003	0.015	0.018
Mental health tests	0.002	0.016	0.019
McDelivery Pakistan	0.0015	0.017	0.018
BlueStacks	0.0021	0.0165	0.0188
Power Searching with Google	0.0037	0.0223	0.0261
TensorFlow playground	0.003	0.0166	0.0197
w3Schools 2018 offline	0.0031	0.0157	0.0189
Touch'D	0.0028	0.0158	0.0186
Average	0.003	0.017	0.02

Abbreviation: ReNaLART, Restricted Natural Language Agile Requirements Testing.

different words appeared in expected and observed outputs. However, ReNaLART, in this case, assigned a fail test verdict because the designed oracle in it is not equipped with handling semantic equivalence of natural language words.

For a user story of Touch'D, a fail test verdict is assigned because the app does not contain the history of outgoing calls in the recent contact list. Therefore, executing the SUT with the test data of this user story, contact names associated with only incoming calls are displayed which instead should also contain contact names of outgoing calls available in the call history of device.

In the Touch'D app if the user changes the last interaction manually then it results in the inability of some contacts to update their last interaction type after receiving a message from some other user. Therefore, ReNaLART assigns a fail test verdict in this case. One of the user stories requires that the user name is not editable but SUT execution shows it to be editable which reveals an error.

For another user story, a test fail verdict is assigned because of wrong computation of time for some contacts. In this case, the string distance functions are not used because numbers identified by the regex pattern are not the same in the observed and expected outputs.

The results show that the oracle proposed in ReNaLART not only detects word level or character level differences in observed and expected outputs but also successfully identifies numeric value differences. It successfully revealed several errors from multiple case studies of different domains.

8 | TEST EFFICIENCY

We can evaluate the efficiency of ReNaLART by estimating the times on an average for test data generation, expected output generation, and test verdict assignment for all case studies which we can use for comparison with other existing approaches.

ReNaLART on an average takes 0.003 s for test data and expected output generation, 0.017 s for verdict assignment and in total takes 0.02 s per user story for all case studies as shown in Table 9.

Moreover, the test cases generated by Yue et al. (2015) take 0.5 min per test case and Wang et al. (2015) take 12 min for test case generation with execution time ranging from 1 to 56 min which shows the time efficiency of ReNaLART.

9 | CONCLUSIONS

In this study, we proposed an approach ReNaLART to automate the test oracle from restricted natural language agile requirements. For this purpose, different templates to write natural language agile requirements as user stories were considered. We chose the Connextra template and applied some other natural language restrictions within it for extracting test data and expected output. For test verdict assignment, the observed and expected outputs were compared by using the regex pattern and different string distance functions. We have also provided a mechanism in ReNaLART to assign an aggregate verdict based on the test verdicts obtained from all the string distance functions. ReNaLART automatically assigned four types of final test verdicts: a pass, a fail, a warning and an inconclusive.

We validated ReNaLART with the help of eight case studies which were: the OLX Pakistan, Mental Health Tests, McDelivery Pakistan, BlueStacks, Power Searching with Google, TensorFlow Playground, w3Schools 2018 offline and Touch'D. The results show that we successfully detected several errors of different types in these case studies which validates our proposed approach ReNaLART.

We also evaluated ReNaLART in terms of test data generation, expected output generation, and verdict assignment time on an average for all case studies. The proposed test oracle in ReNaLART on an average takes 0.02 s for verdict assignment, test data and expected output generation

for a single story. A comparison of ReNaLART with the existing approaches for test case generation shows that it takes less time as compared to existing approaches.

In future, we plan to introduce more string distance functions alongside the existing functions to further strengthen the test oracle in ReNaLART. We also plan to extend ReNaLART to resolve the ambiguities in user stories and expected and observed outputs by instrumenting it with a semantic analyser for natural language to identify ambiguities in natural language text of requirements, expected and observed outputs. This we believe, will also result in far fewer errors in the software end product.

CONFLICT OF INTEREST

The authors declare no conflicts of interest.

ORCID

Muddassar Azam Sindhu https://orcid.org/0000-0002-3411-9224

REFERENCES

Afshan, S., McMinn, P., & Stevenson, M. (2013). Evolving readable string test inputs using a natural language model to reduce human oracle cost. In *2013 IEEE sixth international conference on software testing, verification and validation* (pp. 352–361). Luxembourg, Luxembourg: IEEE. https://doi.org/10.1109/ICST.2013.11

Aichernig, B. K., Hörmaier, K., Lorber, F., Nickovic, D., Schlick, R., Simoneau, D., & Tiran, S. (2014). Integration of requirements engineering and test-case generation via OSLC. In *2014 14th international conference on quality software* (pp. 117–126). Dallas, TX, USA: IEEE. https://doi.org/10.1109/QSIC.2014.13

Alagar, V. S., & Periyasamy, K. (2011). *Specification of software systems*, 2(1–646). London: Springer-Verlag London. https://www.springer.com/gp/book/9780857292766#otherversion=9780857292773.

Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., & Yoo, S. (2015, May). The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, *41*(5), 507–525. https://doi.org/10.1109/TSE.2014.2372785

Baumer, F. S., & Geierhos, M. (2016). Running out of words: How similar user stories can help to elaborate individual natural language requirement descriptions. In G. Dregvaite & R. Damasevicius (Eds.), *Information and software technologies: 22nd international conference, ICIST 2016, Druskininkai, Lithuania, October 13–15, 2016, Proceedings* (pp. 549–558). Switzerland: Springer, Cham.

Bluestacks. (2019). Retrieved from https://www.bluestacks.com/.

Burkhardt, S., & Kärkkäinen, J. (2001). Better filtering with gapped q-grams. In A. Amir (Ed.), *Combinatorial pattern matching: 12th annual symposium, CPM 2001 Jerusalem, Israel, July 1–4, 2001, proceedings* (pp. 73–85). Berlin, Heidelberg: Springer Berlin Heidelberg.

Carrera, Á., Iglesias, C. A., & Garijo, M. (2014, Apr 01). Beast methodology: An agile testing methodology for multi-agent systems based on behaviour driven development. *Information Systems Frontiers*, *16*(2), 169–182. https://doi.org/10.1007/s10796-013-9438-5

Carvalho, G., Falcão, D., Barros, F., Sampaio, A., Mota, A., Motta, L., & Blackburn, M. (2013). Test case generation from natural language requirements based on scr specifications. In *Proceedings of the 28th annual ACM symposium on applied computing* (pp. 1217–1222). New York, NY, United States: Association for Computing Machinery (ACM).

Cha, S.-H. (2007). Comprehensive survey on distance/similarity measures between probability density functions. *International Journal of Mathematical Models and Methods in Applied Sciences*, *1*, 300–307.

Cohn, M. (2004). *User stories applied: For agile software development*, Boston, USA: Addison-Wesley Professional.

Collins, A. (2014). Easyb. Retrieved from http://www.alexecollins.com/5-minute-easyb-bdd-tutorial/.

Cooper, J. (2016). Hamming distance. Retrieved from https://www.maplesoft.com/support/help/Maple/view.aspx?path=StringTools/HammingDistance.

Course builder. (2018). Retrieved from https://edu.google.com/openonline/course-builder/index.html.

Elghondakly, R., Moussa, S., & Badr, N. (2015). Waterfall and agile requirements-based model for automated test cases generation. In *2015 IEEE seventh international conference on intelligent computing and information systems (ICICIS)* (pp. 607–612). Cairo, Egypt: IEEE. https://doi.org/10.1109/IntelCIS.2015.7397285

Extractor. (2017). Retrieved from http://exactor.sourceforge.net/index.html.

Frakes, W. B., & Fox, C. J. (2003). Strength and similarity of affix removal stemming algorithms. *SIGIR Forum*, *37*(1), 26–30. https://doi.org/10.1145/945546.945548

Goffi, A., Gorla, A., Ernst, M. D., & Pezzè, M. (2016). Automatic generation of oracles for exceptional behaviors. In *Proceedings of the 25th international symposium on software testing and analysis* (pp. 213–224). USA: ACM.

Hanssen, G. K., & Haugset, B. (2009). Automated acceptance testing using fit. In *2009 42nd Hawaii international conference on system sciences* (pp. 1–8). Big Island, HI, USA: IEEE. https://doi.org/10.1109/HICSS.2009.83

Hoffman, D. (1998). A taxonomy for test oracles. In *Quality week* (Vol. 98, pp. 52–60). San Francisco, USA: SRI.

Huang, A. (2008). Similarity measures for text document clustering. In *Proceedings of the sixth New Zealand computer science research student conference (NZCSRSC 2008),* (pp. 49–56). Christchurch, New Zealand: NZCSRSC.

Khurshid, S., & Marinov, D. (2004). Testera: Specification-based testing of java programs using sat. *Automated Software Engineering*, *11*(4), 403–434. https://doi.org/10.1023/B:AUSE.0000038938.10589.b9

Landhausser, M., & Genaid, A. (2012). Connecting user stories and code for test development. In *Proceedings of the third international workshop on recommendation systems for software engineering* (pp. 33–37). Piscataway, NJ: IEEE Press. Retrieved from. http://dl.acm.org/citation.cfm?id=2666719.2666727

Last, M., Friedman, M., & Kandel, A. (2004). Using data mining for automated software testing. *International Journal of Software Engineering and Knowledge Engineering*, *14*(04), 369–393. https://doi.org/10.1142/S0218194004001737

Ledru, Y., Petrenko, A., Boroday, S., & Mandran, N. (2012). Prioritizing test cases with string distances. *Automated Software Engineering*, *19*(1), 65–95. https://doi.org/10.1007/s10515-011-0093-0

Leskovec, J., Rajaraman, A., & Ullman, J. D. (2014). *Mining of massive datasets* (2nd ed.). Cambridge, England, UK: Cambridge University Press.

Lucassen, G., Dalpiaz, F., van der Werf, J. M. E. M., & Brinkkemper, S. (2016). The use and effectiveness of user stories in practice. In M. Daneva & O. Pastor (Eds.), *Requirements engineering: Foundation for software quality* (pp. 205–222). Cham, UK: Springer International Publishing.

Lucassen, G., Dalpiaz, F., van der Werf, J. M. E. M., & Brinkkemper, S. (2017). Improving user story practice with the Grimm method: A multiple case study in the software industry. In P. Grünbacher & A. Perini (Eds.), *Requirements engineering: Foundation for software quality: 23rd international working conference, REFSQ 2017, Essen, Germany, February 27–March 2, 2017, proceedings* (pp. 235–252). Cham, UK: Springer International Publishing.

Lucca, G. A. D., Fasolino, A. R., Faralli, F., & Carlini, U. D. (2002). Testing web applications. In *International conference on software maintenance, 2002. proceedings* (pp. 310–319). Montreal, Quebec, Canada: IEEE. https://doi.org/10.1109/ICSM.2002.1167787

Manolache, L. I., & Kourie, D. G. (2001). Software testing using model programs. *Software: Practice and Experience*, *31*(13), 1211–1236. https://doi.org/10.1002/spe.409

Mathur, A. (2013). *Foundations of software testing, 2/e*, (1–728). India: Pearson Education India.

Mayer, J., & Guderlei, R. (2004). Test oracles using statistical methods. In *Testing of Component-Based Systems and Software Quality, Proceedings of {SOQUA} 2004 (First International Workshop on Software Quality) and {TECOS} 2004 (Workshop Testing Component-Based Systems)* (pp. 179–189). Germany: GI. https://dl.gi.de/20.500.12116/28491.

McDelivery Pakistan. (2019). Retrieved from https://play.google.com/store/apps/details?id=pk.com.mcdelivery.

Memon, A. M., Pollack, M. E., & Soffa, M. L. (2000). Automated test oracles for GUIs. *SIGSOFT Softw Eng Notes*, *25*(6), 30–39. https://doi.org/10.1145/357474.355050

Mental Health Tests. (2019). Retrieved from https://play.google.com/store/apps/details?id=org.minddiagnostics.

Navarro, G. (1998). *Approximate text searching (phdthesis)*. Santiago: Department of Computer Science, University of Chile.

Navarro, G. (2001). A guided tour to approximate string matching. *ACM Computing Surveys*, *33*(1), 31–88. https://doi.org/10.1145/375360.375365

Okolnychyi, A., & Fogen, K. (2016). A study of tools for behavior-driven development. In *Full-scale software engineering seminar 2015/16* (pp. 7–12). Germany: RWTH Aachen University.

OLX Pakistan. (2019). Retrieved from https://play.google.com/store/apps/details?id=com.olx.pk.

Pandit, P., & Tahiliani, S. (2015). Agileuat: A framework for user acceptance testing based on user stories and acceptance criteria. *International Journal of Computer Applications*, *120*(10), 16–21.

Peters, D. K., & Parnas, D. L. (1998, Mar). Using test oracles generated from program documentation. *IEEE Transactions on Software Engineering*, *24*(3), 161–173. https://doi.org/10.1109/32.667877

Peterson, D. (2017). Concordion. Retrieved from http://concordion.org/.

Power Searching with Google. (2018). Retrieved from https://coursebuilder.withgoogle.com/sample/.

Rane, P. (2017). *Automatic generation of test cases for agile using natural language processing*. (Unpublished master's thesis). Virginia Tech.

Robeer, M., Lucassen, G., van der Werf, J. M. E. M., Dalpiaz, F., & Brinkkemper, S. (2016). Automated extraction of conceptual models from user stories via NLP. In *2016 IEEE 24th international requirements engineering conference (re)* (pp. 196–205). Beijing, China: IEEE. https://doi.org/10.1109/RE.2016.40

Role-Feature-Reason. (2017). Retrieved from https://www.agilealliance.org/glossary/role-feature/.

Sarmiento, E., do Prado Leite, J. C. S., & Almentero, E. (2014). C&L: Generating model based test cases from natural language requirements descriptions. In *2014 IEEE 1st international workshop on requirements engineering and testing (ret)* (pp. 32–38). Karlskrona, Sweden: IEEE. https://doi.org/10.1109/RET.2014.6908677

Segura, S., Fraser, G., Sanchez, A. B., & Ruiz-Cortés, A. (2016). A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, *42*(9), 805–824.

Shahamiri, S. R., Kadir, W. M. N. W., & Mohd-Hashim, S. Z. (2009). A comparative study on automated software test oracle methods. In *2009 fourth international conference on software engineering advances* (pp. 140–145). Porto, Portugal: IEEE. https://doi.org/10.1109/ICSEA.2009.29

Shahbazi, A., & Miller, J. (2016, April). Black-box string test case generation through a multi-objective optimization. *IEEE Transactions on Software Engineering*, *42*(4), 361–378. https://doi.org/10.1109/TSE.2015.2487958

Shaw, M. (2002). What makes good research in software engineering? *International Journal on Software Tools for Technology Transfer*, *4*(1), 1–7.

Sneed, H. M. (2007). Testing against natural language requirements. In *Seventh international conference on quality software (QSIC 2007)* (pp. 380–387). Portland, OR, USA: IEEE. https://doi.org/10.1109/QSIC.2007.4385524

Sommerville, I., & Sawyer, P. (1997). *Requirements engineering: A good practice guide*, USA: John Wiley & Sons Inc.

Soukoreff, R. W., & MacKenzie, I. S. (2001). Measuring errors in text entry tasks: An application of the levenshtein string distance statistic. In *CHI'01 extended abstracts on human factors in computing systems* (pp. 319–320). New York, NY: ACM. https://doi.org/10.1145/634067.634256

Spock. (2018). Retrieved from http://spockframework.org/.

Stanford, S. (2017). Retrieved from https://github.com/stanfordnlp/CoreNLP/blob/master/data/edu/stanford/nlp/ patterns/surface/stopwords.txt.

Tahchiev, P., Leme, F., Massol, V., & Gregory, G. (2010). *Junit in action* (2nd ed.). Greenwich, CT: Manning Publications Co.

Tensorflow Playground. (2018). Retrieved from https://playground.tensorflow.org/.

Touch'D. (2017). Retrieved from https://bitsol.tech/our-work/.

w3Schools 2018 offline. (2019). Retrieved from https://www.apkonline.net/download-android-apks/app-w3schools-2018-offline

Wang, C., Pastore, F., & Briand, L. (2017). System testing of timing requirements based on use cases and timed automata. In *10th IEEE international conference on software testing, verification and validation (ICST 2017), Tokyo, March 13–18, 2017*, Tokyo, Japan: IEEE.

Wang, C., Pastore, F., Goknil, A., Briand, L., & Iqbal, Z. (2015). Automatic generation of system test cases from use case specifications. In *Proceedings of the 2015 international symposium on software testing and analysis* (pp. 385–396). New York, NY: ACM. https://doi.org/10.1145/2771783.2771812

Yue, T., Ali, S., & Zhang, M. (2015). Rtcm: A natural language based, automated, and practical test case generation framework. In *Proceedings of the 2015 international symposium on software testing and analysis* (pp. 397–408). New York, NY: ACM. https://doi.org/10.1145/2771783.2771799

Zhang, M., Yue, T., Ali, S., Zhang, H., & Wu, J. (2014). A systematic approach to automatically derive test cases from use cases specified in restricted natural languages. In D. Amyot, P. Fonseca i Casas, & G. Mussbacher (Eds.), *System analysis and modeling: Models and reusability* (pp. 142–157). Cham: Springer International Publishing.

AUTHOR BIOGRAPHIES

Maryam Imtiaz Malik received her MPhil degree in computer science from Quaid-i-Azam University, Islamabad, Pakistan in 2017. Currently, she is a PhD student at Quaid-i-Azam University, Islamabad, Pakistan. Her research interests include software testing, natural language processing, and information retrieval systems.

Muddassar Azam Sindhu is an assistant professor of Computer Science Department at Quaid-i-Azam University Islamabad, Pakistan. He received his MSc degree in computer science from Punjab University, Lahore, Pakistan in 2003. He received his Licentiate in Engineering and PhD in Computer Science degrees from Royal Institute of Technology, Stockholm, Sweden in 2011 and 2013, respectively. His research focuses on developing new theories and tools for software testing in both formal and informal paradigms. He is specially interested in testing tool automation for test creation, execution and verdict assignment from requirements expressed formally or informally.

Akmal Saeed Khattak is an assistant professor of Computer Science Department at Quaid i Azam University Islamabad, Pakistan. He received his PhD from University of Liepzig, Germany in 2014. His research interests include information retrieval systems, natural language processing, machine learning and recommender systems. He has remained a reviewer for *NUST Journal of Engineering Sciences* (NJES) and *Journal of the American Society for Information Science and Technology* (JASIST).

Rabeeh Ayaz Abbasi received his PhD from University of Koblenz-Landau, Germany in 2010. He received an HEC-DAAD scholarship for his PhD. During his PhD he worked on extracting and utilizing semantics in social media. Currently, he is working as an associate professor at Computer Science Department, Quaid i Azam University, Islamabad, Pakistan. His current research interests include social media analytics, social network analysis and natural language processing.

Khalid Saleem received his MSc degree in Computer Sciences from Quaid-i-Azam University, Islamabad, Pakistan. He obtained the DEA and PhD degrees in Informatics from the University of Montpellier II, France, in 2005 and 2008, respectively. Since then, he has been with Department of Computer Sciences, Quaid-i-Azam University, where he is actively participating in academic and research activities. His research interests cover the data analytics, machine learning and natural language processing fields and their applications in cross disciplinary domains.

How to cite this article: Malik MI, Sindhu MA, Khattak AS, Abbasi RA, Saleem K. Automating test oracles from restricted natural language agile requirements. *Expert Systems*. 2020;e12608. https://doi.org/10.1111/exsy.12608